

Supplementary Material for: Stateful Selective Maintenance of Two-Level Schwarz Preconditioners for Evolving Sparse Linear Systems

JSR Research Group
Computational Mathematics and High-Performance Engineering Laboratory
Email: jsr-research@domain.org

September 2026

1 Introduction and Overview

This document provides supplementary experimental protocols, complete parameter grids, raw measurement tables, and software environment metadata supporting the main manuscript submitted to *Computer Methods in Applied Mechanics and Engineering* (CMAME). All experimental results reported here and in the main text are fully reproducible using the official open-source repository at <https://github.com/Deepsleepinger/JSR> (Release tag: `v1.2-cmame-final`).

2 Complete Tabular Data for the Five Scientific Pillars

2.1 Pillar 1: Operating Regime Phase Diagram ($\rho \times \Gamma$ Grid)

Table 1 reports the full $4 \times 4 = 16$ parameter cell grid across perturbation spatial radius fraction $\rho \in [0.125, 1.000]$ and perturbation intensity $\Gamma \in [1.0, 16.0]$ on the $24 \times 24 \times 24$ mesh (13,824 DOFs).

Table 1: Full 4×4 parameter grid for the Operating Regime Phase Diagram ($N = 24$, 8 subdomains, 3 steps).

ρ	Γ	T_{full} (s)	T_{JSR} (s)	Speedup	Krylov Iters (Full / JSR)	Max RelRes
0.125	1.0	0.6120	0.5514	1.11×	35.0 / 35.0	4.12×10^{-10}
0.125	2.0	0.6152	0.5542	1.11×	35.3 / 35.3	4.20×10^{-10}
0.125	4.0	0.6201	0.5586	1.11×	35.7 / 35.7	4.35×10^{-10}
0.125	16.0	0.6384	0.5700	1.12×	36.3 / 36.3	4.79×10^{-10}
0.250	1.0	0.6145	0.5586	1.10×	35.0 / 35.0	4.10×10^{-10}
0.250	2.0	0.6190	0.5627	1.10×	35.3 / 35.3	4.18×10^{-10}
0.250	4.0	0.6255	0.5739	1.09×	36.0 / 36.0	4.30×10^{-10}
0.250	16.0	0.6480	0.5891	1.10×	37.0 / 37.0	4.65×10^{-10}
0.500	1.0	0.6210	0.5858	1.06×	35.3 / 35.7	4.15×10^{-10}
0.500	2.0	0.6285	0.5986	1.05×	36.0 / 36.3	4.22×10^{-10}
0.500	4.0	0.6390	0.6144	1.04×	37.0 / 37.3	4.40×10^{-10}
0.500	16.0	0.6720	0.6462	1.04×	38.7 / 39.0	4.72×10^{-10}
1.000	1.0	0.6350	0.6287	1.01×	36.0 / 36.0	4.18×10^{-10}
1.000	2.0	0.6420	0.6484	0.99×	37.0 / 37.0	4.25×10^{-10}
1.000	4.0	0.6580	0.6712	0.98×	38.3 / 38.7	4.45×10^{-10}
1.000	16.0	0.7020	0.7090	0.99×	41.0 / 41.3	4.78×10^{-10}

2.2 Pillar 2: Maintenance-Cost Trade-Off Across Forced Ratios

Table 2 reports the step-by-step performance across forced refresh counts $k_{\text{ref}} \in \{0, \dots, 8\}$ on the $N = 28$ mesh (21,952 DOFs).

Table 2: Detailed breakdown across forced refresh counts vs. adaptive JSR ($N = 28$, 8 subdomains).

k_{ref}	Ratio η	T_{setup} (s)	T_{solve} (s)	T_{total} (s)	Krylov Iters	RelRes
0	0.000	0.0586	0.8639	0.9225	45.3	7.90×10^{-11}
1	0.125	0.0758	0.9319	1.0077	47.3	7.90×10^{-11}
2	0.250	0.0890	0.8374	0.9264	44.0	7.90×10^{-11}
3	0.375	0.1255	0.9314	1.0569	45.0	7.90×10^{-11}
4	0.500	0.1188	0.8707	0.9895	44.7	7.90×10^{-11}
5	0.625	0.1461	0.8818	1.0279	45.0	7.90×10^{-11}
6	0.750	0.1542	0.8631	1.0173	43.7	7.90×10^{-11}
7	0.875	0.1638	0.8504	1.0143	44.0	7.90×10^{-11}
8	1.000	0.1881	0.8711	1.0592	43.7	7.90×10^{-11}
Adaptive JSR	0.250	0.0955	0.8640	0.9594	44.0	7.90×10^{-11}

2.3 Pillar 3: High-Resolution Nanosecond Monitoring Profiling

Table 3 presents timing measurements across 3-D mesh sizes $N \in \{20, 24, 28, 32\}$.

Table 3: Monitoring overhead ratio and drift proxy fidelity across mesh scales.

Mesh N	DOFs	T_{monitor} (ms)	T_{saved} (ms)	η_{mon}	Pearson r	Spearman ρ_s
20	8,000	0.093	33.58	0.28%	1.0000	0.8333
24	13,824	0.144	51.41	0.28%	1.0000	0.9762
28	21,952	0.216	110.54	0.20%	1.0000	0.9762
32	32,768	0.222	147.04	0.15%	1.0000	0.7857

2.4 Pillar 4: Truncation Parameter α Sensitivity Sweep

Table 4 provides the full parameter sweep for $\alpha \in [0.75, 0.99]$.

Table 4: Cumulative-drift truncation parameter α sweep ($N = 28$, 8 subdomains).

α	k_{sel}	T_{setup} (s)	T_{solve} (s)	T_{total} (s)	Krylov Iters	RelRes
0.75	2.0	0.0914	0.8352	0.9266	44.0	7.90×10^{-11}
0.80	2.0	0.0884	0.8439	0.9323	44.0	7.90×10^{-11}
0.85	2.0	0.0899	0.8367	0.9267	44.0	7.90×10^{-11}
0.90	2.0	0.0906	0.8427	0.9333	44.0	7.90×10^{-11}
0.95	2.7	0.1002	0.8417	0.9419	44.3	1.50×10^{-10}
0.97	3.7	0.1157	0.8467	0.9624	44.7	2.57×10^{-10}
0.98	4.7	0.1294	0.8475	0.9768	44.3	2.32×10^{-10}
0.99	5.7	0.1510	0.8361	0.9871	44.0	2.00×10^{-10}

2.5 Pillar 5: Mesh Resolution Sweep and Power-Law Scaling

Table 5 reports the execution times across $N \in \{16, \dots, 48\}$.

Table 5: 3-D mesh sweep and local factorization scaling ($N \in [16, 48]$, 8 subdomains).

Mesh N	Global DOFs	Sub DOFs	$T_{\text{fact}}^{\text{sub}}$ (s)	Setup Frac	T_{full} (s)	T_{JSR} (s)	Speedup
16	4,096	729	0.0027	18.39%	0.1872	0.1670	1.12×
20	8,000	1,331	0.0049	18.92%	0.3451	0.3030	1.14×
24	13,824	2,197	0.0090	17.06%	0.6597	0.5857	1.13×
28	21,952	3,375	0.0152	17.78%	1.0154	0.8829	1.15×
32	32,768	4,913	0.0249	18.96%	1.5877	1.3907	1.14×
36	46,656	6,859	0.0662	27.98%	2.3640	2.0328	1.16×
48	110,592	15,625	0.1956	28.44%	6.4988	5.0834	1.28×

2.6 Subdomain Granularity Study ($N_{\text{sub}} \in \{8, 27, 64\}$)

Table 6 details the partition resolution scaling on $N = 32$ (32,768 DOFs).

Table 6: Subdomain granularity scaling across $N_{\text{sub}} \in \{8, 27, 64\}$ ($N = 32$, 3 time steps).

N_{sub}	Grid	Sub DOFs	$ S_t /N_{\text{sub}}$	$T_{\text{setup}}^{\text{full}}$ (s)	$T_{\text{setup}}^{\text{JSR}}$ (s)	$T_{\text{solve}}^{\text{full}}$ (s)	$T_{\text{total}}^{\text{full}}$ (s)	$T_{\text{total}}^{\text{JSR}}$ (s)	Speedup
8	$2 \times 2 \times 2$	4,913	7.3/8 (91.7%)	0.2734	0.2518	0.9880	1.2620	1.2786	0.99×
27	$3 \times 3 \times 3$	1,728	10.0/27 (37.0%)	0.2660	0.1573	1.3384	1.6048	1.5433	1.04×
64	$4 \times 4 \times 4$	857	20.3/64 (31.8%)	0.2878	0.1584	1.7194	2.0076	1.8960	1.06×

3 Exact Reproduction Command Reference

To independently reproduce all tables and figures from clean checkout:

```
# 1. Clone repository at the submission freeze tag
git clone https://github.com/Deepsleepinger/JSR.git
cd JSR
git checkout v1.2-cmame-final

# 2. Run unit tests
python -m unittest discover tests/

# 3. Reproduce Flagship 48^3 (110k DOFs) benchmark
python benchmarks/run_cmame_flagship_48.py

# 4. Reproduce Subdomain Granularity Study (8 -> 27 -> 64)
python benchmarks/run_cmame_subdomain_scaling.py

# 5. Reproduce Complete Five Pillars Verification Suite
python benchmarks/run_cmame_5pillars.py --pillar all
```